5-4 · Spark DataFrame과 SparkSQL 데이터 가공 · 이론 1H

DataFrame과 SparkSQL 동작 원리

API 와 SQL 이 같은 곳으로 수렴하는 이유

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① PySpark DataFrame 문법을 Pandas 문법과 짝지어 말할 수 있다

② API 와 SQL 이 같은 실행 계획으로 번역되는 흐름을 설명한다

③ F.col 로 컬럼을 가리키고, 조인 키를 문자열로 주는 이유를 안다

④ 조인 how 다섯 가지와 두 조인 방식이 정해지는 기준을 안다

다음 1.5H 실습에서 orders·products 두 표로 조인·집계까지 직접 해 봅니다

1. 분산된 표

Pandas 와 PySpark — 이름이 거의 그대로

하려는 일	Pandas (3주차)	PySpark
열 고르기	df[['a','b']]	df.select("a", "b")
조건 필터	df[df.a > 10]	df.filter(F.col("a") > 10)
파생 열	df.assign(c=...)	df.withColumn("c", ...)
그룹 집계	df.groupby('g').agg(...)	df.groupBy("g").agg(...)
조인	a.merge(b, on='k')	a.join(b, on="k")
정렬	df.sort_values('a')	df.orderBy("a")
보기	df.head()	df.show()

엑셀 시트가 여러 컴퓨터에 조각조각 나뉘어 있다 — 표 전체를 다루듯 쓰고, 조각 처리는 Spark 가 맡는다

다른 점은 지연 평가(action 을 불러야 실행)와 F.col(...) 로 컬럼을 가리킨다는 것 정도

2. 수렴

API 든 SQL 이든 — 같은 논리 계획으로 번역된다



한국어로 말하든 영어로 말하든 같은 주문서가 주방에 전달된다

그래서 성능 차이가 아니라 표현의 취향·상황 문제 — 오늘 explain() 출력을 직접 대조합니다

2. 수렴

API 와 SQL — 언제 무엇을 고를까

DataFrame API

여러 단계를 변수로 이어 붙이는 파이프라인

- 중간 결과에 이름을 붙여 재사용
- 함수로 쪼개기 쉬워 단위 테스트에 유리
- 3주차 Pandas 감각이 그대로

VS

SparkSQL

분석가와 공유하는 집계 · 한눈에 읽히는 쿼리

- 한 문장으로 읽히는 집계 로직
- 실무에서는 한 스크립트에 섞어 쓴다
- 3-4 의 DuckDB↔Pandas 왕복과 같은 발상

컬럼을 가리키는 세 가지 방법 — F.col 이 기본

1. **(1) F.col("amount") > 100000**
   가장 안전 — 권장
2. **(2) df["amount"] > 100000**
   그 DataFrame 소속임이 분명 — 조인 뒤 같은 이름이 양쪽에 있을 때
3. **(3) "amount > 100000"**
   SQL 문자열 — 짧고 읽기 좋다

기본값

F.col(...)

> **from pyspark.sql import functions as F — 파일 첫 줄에 한 번**

4. 조인

조인 how 다섯 가지 — 무엇이 남는가

how	남는 것	주의
inner (기본)	양쪽 다 있는 키만	한쪽에 없으면 행이 사라진다
left	왼쪽 전부	파이프라인의 기본 선택
right / full	오른쪽 / 양쪽 전부	—
left_anti	오른쪽에 없는 왼쪽 행만	「매칭 안 된 것 찾기」에 유용
left_semi	오른쪽에 있는 왼쪽 행만	컬럼은 왼쪽 것만 남는다

오른쪽 키가 중복이면 행이 늘고 합계가 부풀려진다 — Spark 에는 validate 옵션이 없으니 조인 전후 count() 를 직접 비교

4. 조인

조인에서 컬럼이 충돌할 때 — 두 가지 처방

키를 문자열로

결과에 product_id 가 한 번만 남는다

- on="product_id" — 같은 이름일 때
- a["pid"] == b["product_id"] — 다른 이름
- 이름이 다르면 두 컬럼이 다 남는다

VS

별칭(alias)으로 출처를

같은 이름이 양쪽에 있을 때 o. · p. 로 구분

- ao, bo = a.alias("o"), b.alias("p")
- F.col("o.pid") == F.col("p.product_id")
- select("o.pid", "p.name") 로 출처 명시

5. 조인 방식

조인 방식은 Spark 가 추정한 크기로 정한다

Broadcast Hash Join

한쪽이 작다고 Spark 가 알 때(기본 10MB 이하)

- 작은 쪽을 모든 노드에 복사 — 조인 쪽 셔플 없음
- Iceberg 에서 읽은 표는 크기를 알아 힌트 없이
- F.broadcast(products) 로 「작다」고 알려 준다

VS

Sort Merge Join

양쪽 다 크거나 — 크기를 모를 때

- 양쪽을 키로 재배치 — 조인 양쪽에 Exchange
- createDataFrame 표는 크기를 몰라 아주 크다고 본다
- 세 줄짜리 products 도 여기로 간다(실측)

정리

이 1시간에서 꼭 가져갈 것

1. PySpark 의 DataFrame 문법은 Pandas 와 거의 같은 이름이다
2. API 와 SQL 은 같은 논리 계획으로 번역된다 — 취향·상황으로 고른다
3. 컬럼은 F.col 로 가리키는 것이 가장 안전하다
4. 조인 키를 문자열로 주면 중복 컬럼이 안 생긴다
5. 조인 후에는 행 수를 직접 확인한다 — Spark 에는 validate 가 없다
6. 조인 방식은 추정 크기로 정해진다 — 모르면 F.broadcast 로 알려 준다

5-4 · 이론 1H 마무리

다음은 DataFrame API 실습 (1.5H)

orders·products 두 표를 만들어 타입 변환 → 조인 → 파생 → 필터 → 집계 → 윈도까지 체인으로 잇고 explain() 을 봅니다

- ✓ **createDataFrame 으로 orders 5행 · products 3행**
- ✓ **to_date 로 타입 바로잡기**
- ✓ **join → withColumn → filter → groupBy**
- ✓ **explain() — SortMergeJoin 과 F.broadcast**

준비할 것

실습점검으로 클러스터 켜기 · bash lake-pyspark.sh